Test Automation Frameworks for CI/CD

Day 2 · Module 2 · 2 Hours

SDET / DevOps Engineer

Intermediate

45% Theory / 55% Hands-on

TOOLS

Selenium / Playwright / Cypress

TestNG / JUnit 5 / PyTest

Maven / Gradle

LEARNING OUTCOMES

What You Will Achieve Today

Design automation frameworks for CI execution

Apply modular, maintainable architecture patterns suitable for pipeline-first test execution.

Apply modular and maintainable test design

Use data-driven, keyword-driven, and CI-friendly design patterns across TestNG, JUnit 5, and PyTest.

Integrate test frameworks with build tools

Wire Maven Surefire/Failsafe and Gradle test tasks to run tests from the command line and in CI.

Automation Framework Architectures

Linear / Record-and-Playback Use when: Quick PoC only	PRO	Zero setup
	CON	Brittle, not maintainable, no reuse
Modular Framework Use when: Small–mid teams	PRO	Reusable functions, clean separation
	CON	Can grow messy without discipline
Data-Driven Framework Use when: Many test variants	PRO	Tests + data decoupled; easy scaling
	CON	Needs external data layer (CSV/Excel/DB)
Keyword-Driven Framework Use when: Mixed skill teams	PRO	Non-devs can add tests
	CON	Heavy upfront keyword library cost
Hybrid Framework Use when: Enterprise / CI-first	PRO	Best of modular + data-driven + POM
	CON	Highest setup complexity

FRAMEWORK COMPARISON

TestNG vs JUnit 5 vs PyTest — Choosing for CI

TestNG

v7.x · Java

KEY STRENGTHS

- ✓ XML suite config (testng.xml)
- ✓ Built-in parallel execution
- ✓ Group, sequence & dependency control
- ✓ Rich HTML reports out of the box
- ✓ Strong for large Selenium suites

```
CI:  mvn test  (via Surefire plugin)
```

WATCH OUT

- ⚠ XML config can become complex
- ⚠ Less unit-test-friendly than JUnit

JUnit 5

v5.14.x · Java

KEY STRENGTHS

- ✓ Modular (Platform / Jupiter / Vintage)
- ✓ Parameterized tests via @ParameterizedTest
- ✓ Extension model replaces Rules
- ✓ Default in Spring Boot Test & most Java stacks
- ✓ Native IDE & Maven/Gradle support

```
CI:  mvn test  /  ./gradlew test
```

WATCH OUT

- ⚠ No built-in HTML report (add Allure)
- ⚠ Parallel config needs extra setup

PyTest

v9.0.x · Python

KEY STRENGTHS

- ✓ Simple assert rewriting — no boilerplate
- ✓ 800+ plugins (pytest-xdist, pytest-cov)
- ✓ Powerful fixture injection & scoping
- ✓ Supports Python 3.10–3.14+
- ✓ Dominant Python test framework for API & service testing

```
CI:  pytest --junitxml=report.xml
```

WATCH OUT

- ⚠ Fixture scoping can confuse newcomers
- ⚠ Async test support needs pytest-asyncio

Data-Driven & Keyword-Driven Testing

Data-Driven Testing (DDT)

- Test logic and test data separated into distinct layers
- Same test executes N times with N data rows
- Data sources: CSV, Excel, JSON, DB, or @ParameterizedTest
- CI executes all variants automatically — no code duplication

JUnit 5 — @ParameterizedTest

```
@ParameterizedTest
@CsvSource({"user1,pass1", "user2,pass2"})
void loginTest(String user, String pwd) {
    loginPage.login(user, pwd);
    assertThat(dashboard.isDisplayed());
}
```

Keyword-Driven Testing (KDT)

- Tests expressed as readable action keywords in a table
- Implementation hidden behind keyword library functions
- Non-developers can write or modify test scenarios
- Framework maps keywords → Java/Python method calls

Keyword Table (Robot Framework style)

Keyword	Argument 1	Argument 2
Open Browser	https://app	chrome
Enter Text	username_fld	testuser
Click Element	login_btn	
Verify Title	Dashboard	

CI-Friendly Test Design Principles

Tests must be Independent & Idempotent

No shared mutable state between tests. Each test sets up its own data and cleans up after itself. Avoids order-dependent failures in parallel CI runs.

Optimise for fast feedback — unit tests should be very fast

Slow tests drag CI feedback loops. Isolate I/O, mock external dependencies, and use fixtures that don't hit networks or databases. Aim to keep the full unit suite under a few minutes in CI.

Zero tolerance for flaky tests in CI

A test that passes/fails intermittently is worse than no test. Quarantine flaky tests, fix root cause (timing, shared state, env), then reinstate.

Environment configuration via properties/env-vars

No hardcoded URLs, credentials, or paths. Use application.properties (Maven), gradle.properties, or .env files — inject at pipeline runtime.

Tag & group tests for selective CI execution

Use TestNG groups, JUnit 5 @Tag, or pytest marks to run smoke, regression, or integration subsets on different pipeline triggers.

Maven & Gradle — Wiring Tests to CI

Apache Maven

pom.xml — Surefire (unit tests)

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-surefire-plugin</artifactId>
  <version>3.5.5</version>
  <configuration>
    <includes>**/*Test.java</includes>
    <parallel>methods</parallel>
    <threadCount>4</threadCount>
```

Common Maven CLI commands

```
# Run all unit tests
mvn test

# Run specific test class
mvn test -Dtest=LoginTest
```

Gradle

build.gradle — test task (JUnit 5)

```
test {
  useJUnitPlatform()
  maxParallelForks = Runtime.runtime
    .availableProcessors().intdiv(2) ?: 1
  testLogging {
    events 'passed','failed','skipped'
    showStandardStreams = true
  }
```

Common Gradle CLI commands

```
# Run all tests
./gradlew test

# Run specific test class
./gradlew test --tests LoginTest
```

DEMO

Demo: Framework Setup → CLI Execution → Reports

01 Framework Project Structure Walkthrough

- Open a sample Maven/Gradle project in IDE
- Walk through: src/test/java, resources, pom.xml / build.gradle
- Identify package structure: pages, tests, utils, data
- Show how Page Object Model separates UI logic from test logic

02 Executing Tests via Command Line

- Run full suite: mvn test / ./gradlew test
- Run tagged subset: mvn test -Dgroups=smoke
- Inspect console output — pass/fail counts, assertion errors
- Observe exit code: 0 = success, non-zero = failure → CI fails build

03 Generating Test Reports

- Maven Surefire generates XML in target/surefire-reports/
- Gradle test reports in build/reports/tests/test/index.html
- Add Allure to JUnit 5: allure generate / allure serve
- CI systems (Jenkins, GitHub Actions) publish JUnit XML natively

CI-Ready Framework — Project Skeleton

Maven project structure

```
my-automation-framework/
├── pom.xml                    # deps + Surefire/Failsafe
├── src/
│   ├── main/java/
│   │   └── pages/             # Page Object classes
│   └── test/
│       ├── java/
│       │   ├── tests/         # @Test classes (TestNG / JUnit5)
│       │   ├── utils/         # Helpers, base classes
│       │   └── data/          # Data providers / factories
│       └── resources/
│           ├── testng.xml     # Suite config (TestNG)
│           ├── config.properties
│           └── testdata/      # CSV / JSON data files
└── .github/workflows/ci.yml  # GitHub Actions pipeline
```

Design Notes

Page Object Model (POM)

Each UI page = one class. Tests call page methods, never raw locators. Reduces duplication; locator changes in one place.

Shared Setup via Lifecycle Hooks

Common in Java frameworks: shared driver init and config via base classes. In JUnit 5, use @BeforeEach/@BeforeAll; TestNG uses @BeforeSuite; PyTest uses session-scoped fixtures. Choose the mechanism that fits your framework.

Config properties

BASE_URL, BROWSER, TIMEOUT loaded from config.properties or env-vars. Never hardcode in test code.

Test Data Layer

CSV/JSON data files or @DataProvider methods. Keep test data version-controlled alongside test code in the repo.

HANDS-ON LAB

Lab: Set Up, Configure & Execute

45% Theory | 55% Hands-on

Task 1

Working framework imported

Set Up a Basic Automation Framework

1. Clone starter repo: git clone <course-repo>
2. Import into IDE as Maven or Gradle project
3. Verify dependencies resolve: mvn dependency:resolve
4. Review project structure — identify each folder's role

Task 2

Configured pom.xml or build.gradle

Configure Build Tool Integration

1. Add Surefire (Maven) or useJUnitPlatform() (Gradle) config
2. Set parallel thread count to 2 in pom.xml / build.gradle
3. Add a test group/tag: @Tag("smoke") [JUnit 5] or groups="smoke" [TestNG]
4. Run subset: mvn test -Dgroups=smoke [TestNG] or configure includeTags in pom.xml [JUnit 5]

Task 3

Execution logs + report screenshot

Execute Tests Locally via CLI & View Report

1. Run full suite from terminal: mvn test / ./gradlew test
2. Run smoke subset: mvn test -Dgroups=smoke
3. Locate and open the HTML report in browser
4. Introduce a deliberate failure — observe exit code and log

Day 2 Deliverables & Key Takeaways

📦 Deliverables You Take Away

- Automation Framework Skeleton (Maven/Gradle project)
- Build & Execution Configuration Files (pom.xml / build.gradle)
- Test Execution Report Screenshot + CLI log

💡 Key Takeaways

1. Hybrid frameworks (modular + data-driven + POM) suit CI-first projects best
2. TestNG excels for Selenium suites; JUnit 5 dominates Java unit testing; PyTest is the dominant Python testing framework
3. Maven Surefire/Failsafe and Gradle test task are the standard CI integration points
4. CI-friendly tests: independent, fast, no hardcoded config, tagged for selective runs
5. Test reports (JUnit XML) are the contract between your framework and your CI system

Tomorrow — Day 3: Version Control & Test Code Management · Git workflows, branching strategies, PR best practices

Hands-on: Push automation code to repo · create feature branch · raise and merge a PR

RECAP QUIZ

Module 2 – Quick Knowledge Check

Q1. Which Maven plugin runs unit tests, and which runs integration tests?

- maven-surefire-plugin runs unit tests (mvn test). maven-failsafe-plugin runs integration tests — it does not fail the build during integration-test, allowing post-integration-test cleanup to run first; results are checked and the build fails (if needed) only during the verify phase.

Q2. Name two key differences between TestNG and JUnit 5.

- Any two: TestNG uses testng.xml for suite config; JUnit 5 uses a modular Extension model. TestNG has built-in parallel execution; JUnit 5 needs config. TestNG has built-in HTML reports; JUnit 5 requires Allure or similar.

Q3. What is data-driven testing and why does it reduce code duplication?

- DDT separates test logic from test data — the same test method runs repeatedly with different inputs (e.g., via @ParameterizedTest or @DataProvider). Adding new scenarios only requires a new data row, not new test code.

Q4. What exit code does a failed test suite return, and why does that matter in CI?

- A non-zero exit code (typically 1). CI systems interpret a non-zero exit as build failure, automatically blocking the pipeline and notifying the team — the core quality gate mechanism.